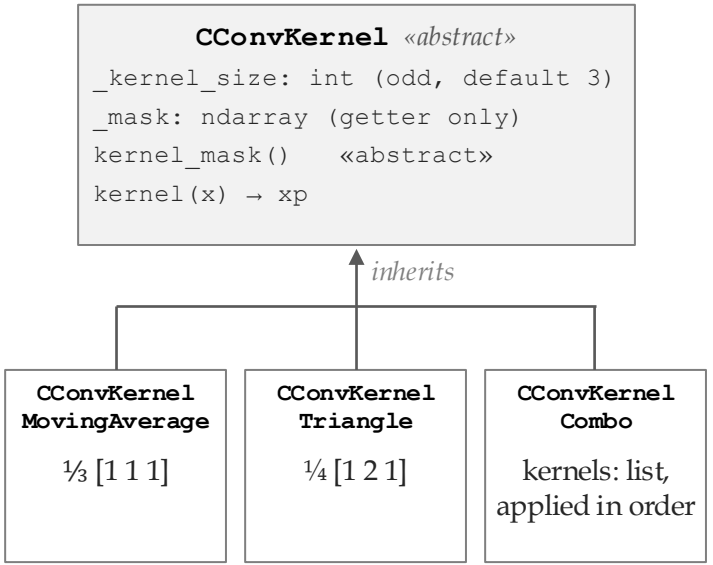
# Python Practical Session on 1D and 2D Convolutional Kernels

*Instructor*
**Battista Biggio**

M.Sc. in Computer Engineering, Cybersecurity and Artificial Intelligence
University of Cagliari, Italy

# Coding Exercise

- Implement the package *conv_1d_kernels* (class diagram on the right)
- `kernel_mask()` builds the mask from *kernel_size*:
  - once in the constructor, again only if *kernel_size* changes (setter)
- `kernel(x)` slides the mask over the signal $x$; the first and last $(K-1)/2$ elements stay unchanged
- Main program: one MNIST digit per class, filtered by each kernel, plots saved as PDF
- Today: what these kernels compute and why, in 1D and in 2D



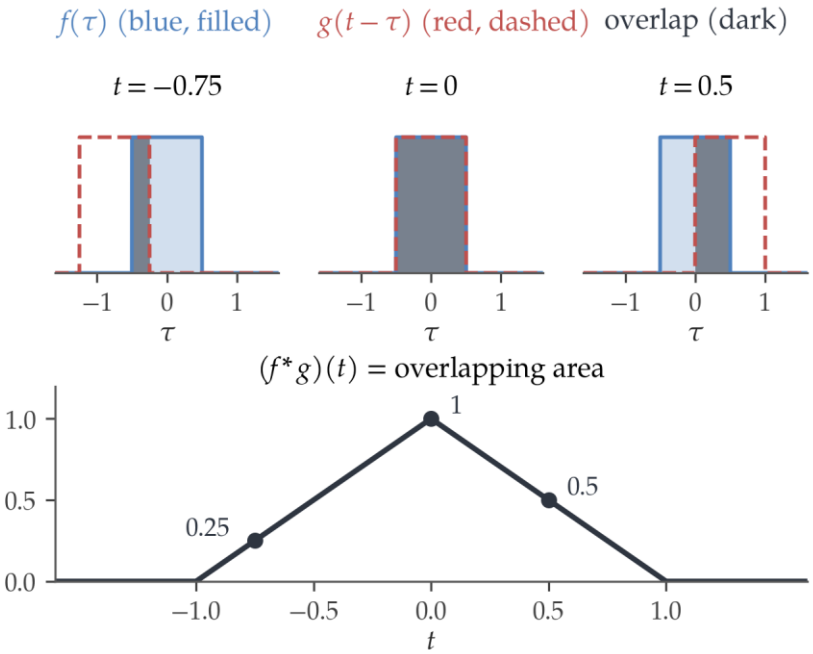
# Convolution: Theory

From the convolution integral to the discrete sum,
its properties, and the frequency domain

# Convolution: the Continuous Case

$$(f * g)(t) = \int_{-\infty}^{+\infty} f(\tau)\, g(t-\tau)\, d\tau$$

- Flip $g$, shift it by $t$, multiply by $f$, integrate: the area of the overlap
- One value per shift $t$: the result is a new function of $t$
- Two boxes give a triangle (right): keep it in mind for later
- Any linear, shift-invariant system can be expressed as $y = x * h$
  - $h$ is the impulse response / kernel: a **filter** is fully described by its kernel

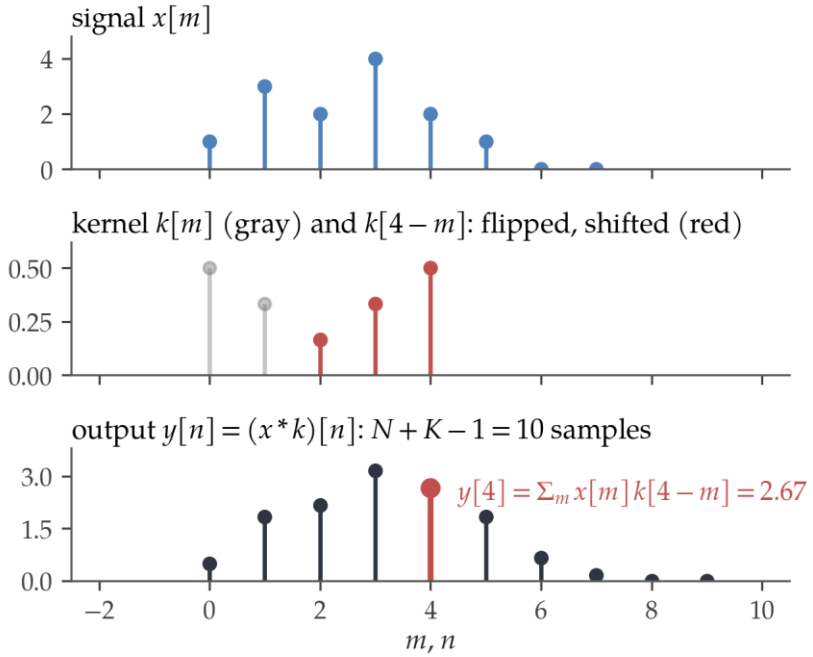


# Discrete Convolution

$$(x * k)[n] = \sum_{m=-\infty}^{+\infty} x[m]\, k[n-m]$$

$$= \sum_{j=-h}^{h} k[j]\, x[n-j], \qquad h = \frac{K-1}{2}$$

- Same recipe as the integral: flip, shift, multiply, sum
- A finite kernel of odd size $K$ is centred in zero: $j = -h, \ldots, h$
- Full convolution of $N$ samples with $K$ weights: $N + K - 1$ samples




Note: the plot indexes the kernel from 0 to $K - 1$ (as NumPy does), so $k[4 - m]$ spans $m = 2 \ldots 4$. With the centred kernel ($-h \ldots h$), it spans $3 \ldots 5$ and $y[n]$ shifts left by $h$.

# Properties of Convolution

**Commutativity**

$$x * k = k * x$$

→ The order of the kernels in a combo does not change the output (borders aside)

**Associativity**

$$(x * k_1) * k_2 = x * (k_1 * k_2)$$

→ A combo of kernels is one single kernel $k_1 * k_2$, of length $K_1 + K_2 - 1$

**Distributivity**

$$x * (k_1 + k_2) = x * k_1 + x * k_2$$

→ Kernels can be added

**Identity**

$$x * \delta = x, \qquad \delta * k = k$$

→ Filtering an impulse returns the kernel: a quick test for any implementation

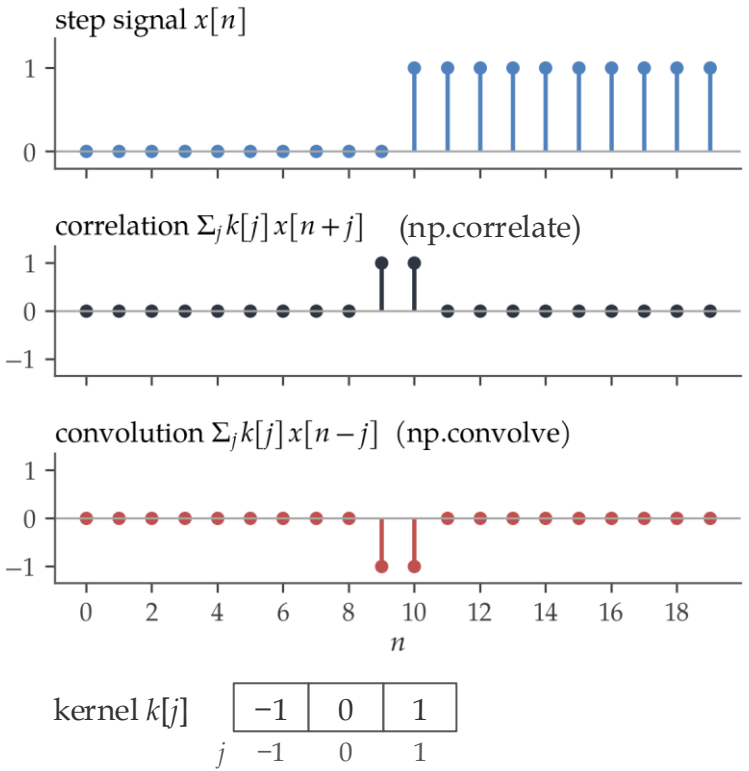
# Convolution vs. Correlation

$$\text{convolution:} \quad (x * k)[n] = \sum_{j=-h}^{h} k[j]\, x[n-j]$$

$$\text{correlation:} \quad (x \star k)[n] = \sum_{j=-h}^{h} k[j]\, x[n+j]$$

$$x * k = x \star \tilde{k}, \qquad \tilde{k}[j] = k[-j]$$

- CNN layers compute a correlation: no flip
- Symmetric masks, $k[j] = k[-j]$: same result (moving average, triangle)
- Asymmetric masks, e.g. [−1 0 1]: the sign flips (right)
- `np.convolve` flips the kernel, `np.correlate` does not



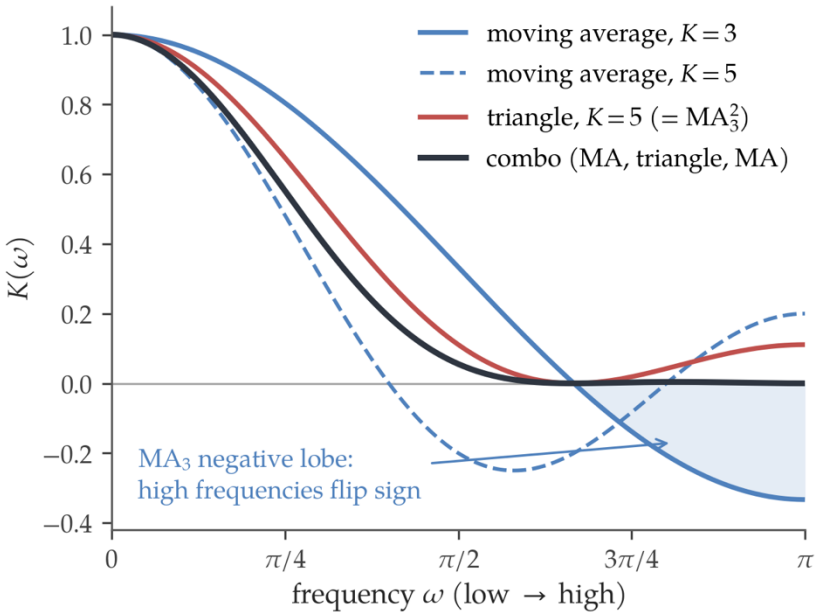
# The Convolution Theorem

$$\mathcal{F}\{f * g\} = \mathcal{F}\{f\} \cdot \mathcal{F}\{g\}$$

$$y = x * k \iff Y(\omega) = X(\omega)\,K(\omega)$$

$$K(\omega) = \sum_{m=-h}^{h} k[m]\, e^{-\mathrm{j}\omega m}, \quad \mathrm{j} = \sqrt{-1}$$

- Convolution in time (or space) = product in frequency
- $K(\omega)$ says how much of each frequency survives: smoothing kernels are low-pass filters
- Box $*$ box = triangle $\Leftrightarrow$ $K_{\text{box}}(\omega)^2 = K_{\text{tri}}(\omega) \geq 0$
- With the FFT: $O(N \log N)$, useful for large kernels (`scipy.signal.fftconvolve`)




Frequency in Hz: $f = \omega \cdot f_s / 2\pi$, with $f_s$ the sampling rate
$0 \to 0$ Hz, $\pi/4 \to f_s/8$, $\pi/2 \to f_s/4$, $\pi \to f_s/2$ (Nyquist)

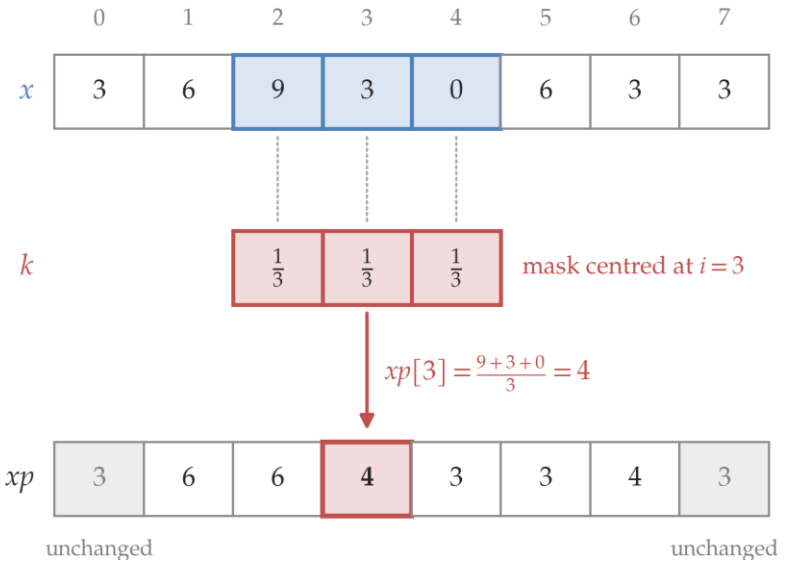
# 1D Convolutional Kernels

Sliding a mask over signals
and over flattened images

# Sliding a Mask over a Signal

$$xp[i] = \sum_{j=-h}^{h} k[j]\, x[i+j], \qquad h = \frac{K-1}{2}$$

- Centre the mask at $x[i]$, multiply element by element, sum: one output value
- Repeat for every $i$: the mask slides along the signal
- $K$ is odd: the mask has a centre and $h$ elements on each side
- The first and last $h$ values are left unchanged
- A correlation: for symmetric masks, the same as a convolution

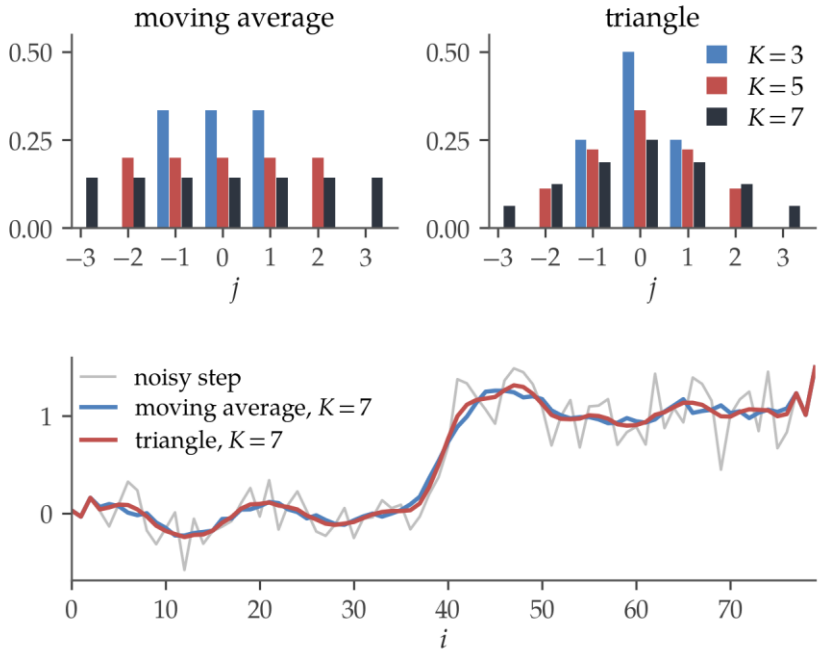


# Smoothing Kernels: Moving Average and Triangle

$$\text{moving average:} \quad k[j] = \frac{1}{K}$$

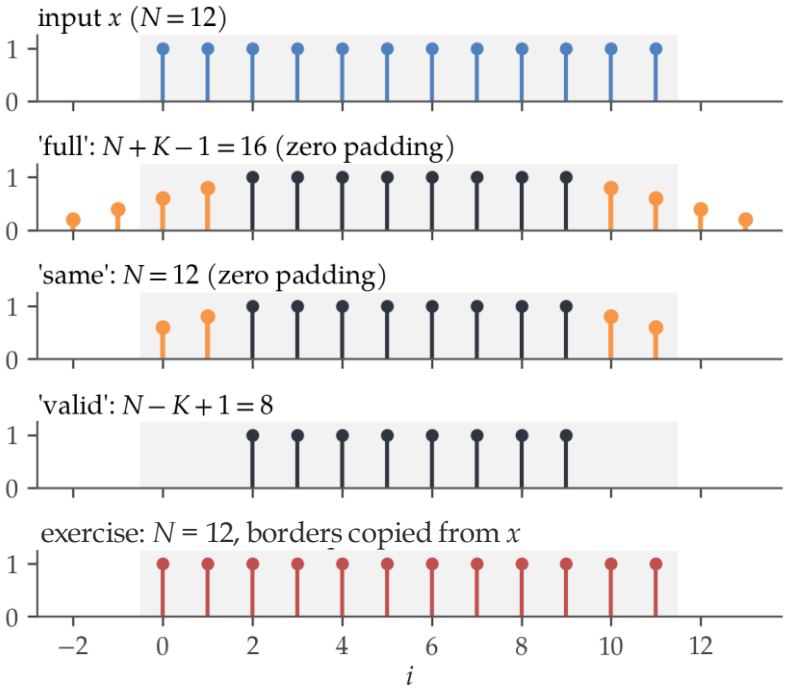
$$\text{triangle:} \quad k[j] = \frac{h+1-|j|}{(h+1)^2}$$

- Both masks sum to one:
  - constant signals are left unchanged
  - outputs are weighted averages and stay in the input range (gray levels)
- Larger $K$: stronger smoothing, and blurrier edges
- The triangle weighs closer samples more
- Examples with $K = 5$:
  - [1 1 1 1 1]/5 and [1 2 3 2 1]/9



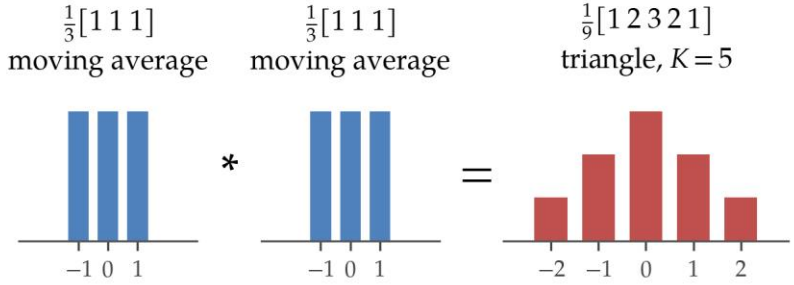
# Handling the Borders

- Near the borders the mask needs samples that do not exist
- `np.convolve` modes:
  - `'full'`: zero padding, $N + K - 1$ outputs
  - `'same'`: zero padding, $N$ outputs
  - `'valid'`: complete windows, $N - K + 1$ outputs
- Exercise: the valid part, plus the first and last $h$ samples copied from $x$
- Zero padding pulls the borders down (orange), so `mode='same'` differs from the exercise version
  - other paddings: reflect, nearest, wrap (`scipy.ndimage`)



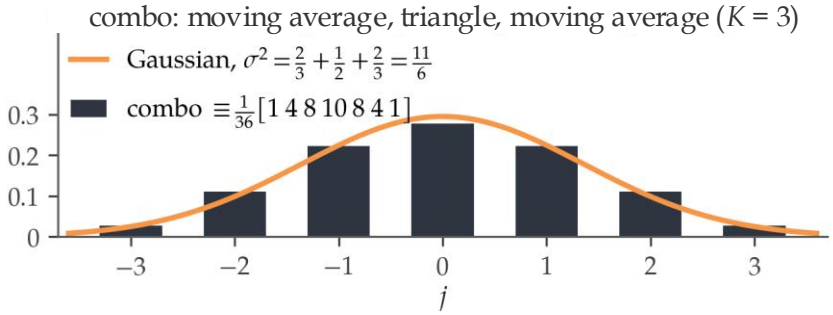
# Composing Kernels: the Combo

- Applying $k_1$ and then $k_2$ = applying the single kernel $k_1 * k_2$
- Two moving averages ($K = 3$) give exactly the triangle with $K = 5$
- The exercise combo is one bell-shaped 7-tap kernel: repeated smoothing tends to a Gaussian



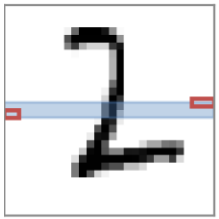

**Impulse test** with your own classes:

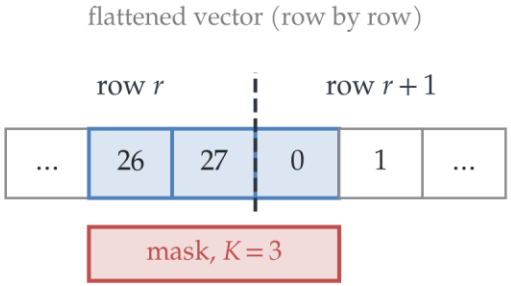
```
ma = CConvKernelMovingAverage()
tri = CConvKernelTriangle()
combo = CConvKernelCombo([ma, tri, ma])
delta = np.zeros(15)
delta[7] = 1  # impulse
print(36 * combo.kernel(delta)[4:11])
# [ 1.  4.  8. 10.  8.  4.  1.]
```

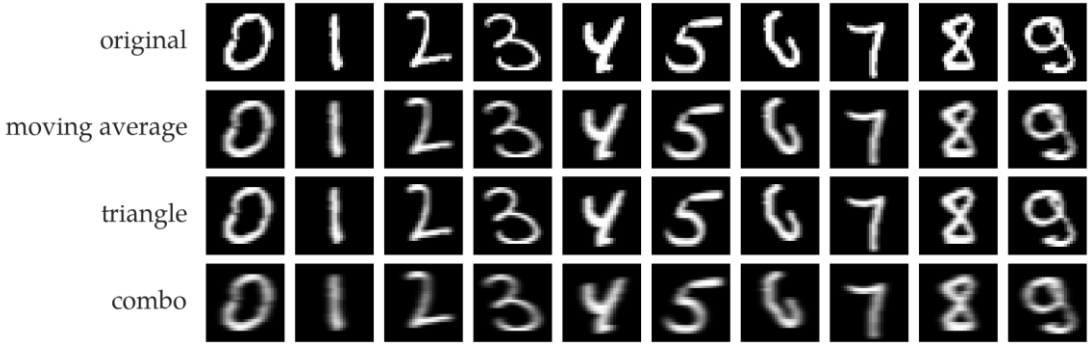


# 1D Kernels on a Flattened Image

- Images are flattened row by row (784 pixels)
- A 1D kernel mixes horizontal neighbours only
- At the end of a row, the mask spills into the next row
- Harmless on MNIST (black borders), but images need 2D kernels






Exercise output, $K = 3$.

Vertical strokes get wider, horizontal strokes barely change: the blur is along rows only.

# 2D Convolutional Kernels

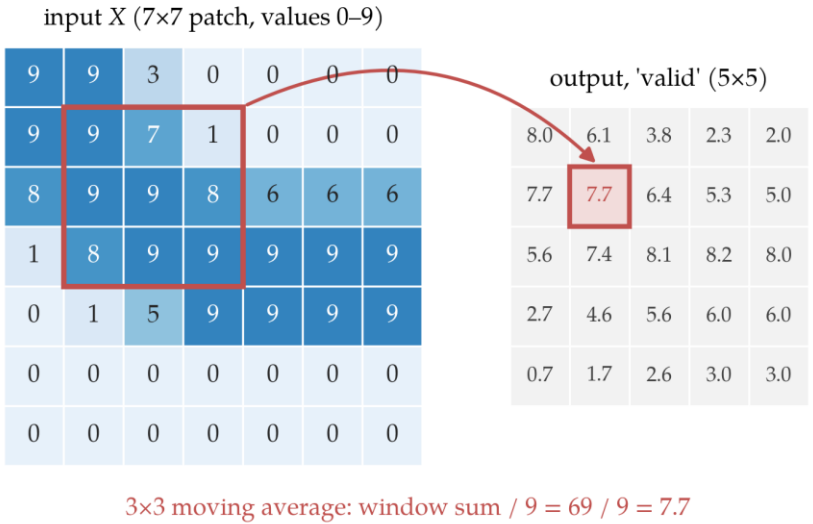
From signals to images

# 2D Convolution

$$(X * K)[i,j] = \sum_{u=-h}^{h} \sum_{v=-h}^{h} K[u,v]\, X[i-u,\, j-v]$$

$$(X \star K)[i,j] = \sum_{u=-h}^{h} \sum_{v=-h}^{h} K[u,v]\, X[i+u,\, j+v]$$

- A $K \times K$ mask slides over rows and columns
- As in 1D, practice (and CNNs) uses the correlation $\star$
- Borders on four sides: the valid output is $(H - K + 1) \times (W - K + 1)$
- Cost: $K^2$ multiply-adds per pixel



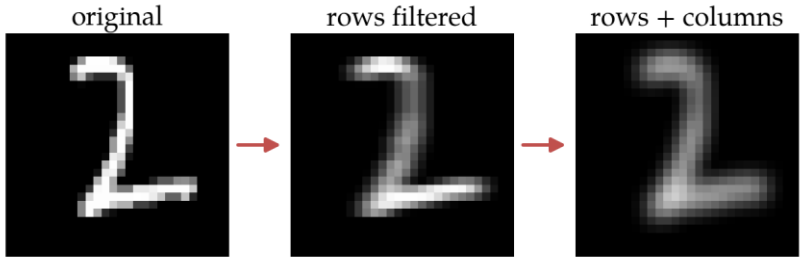

3×3 moving average: window sum / 9 = 69 / 9 = 7.7

# Separable Kernels

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix} = \frac{1}{4}\begin{bmatrix}1\\ 2\\ 1\end{bmatrix}\frac{1}{4}\begin{bmatrix}1 & 2 & 1\end{bmatrix}$$

$$X * \left(k_{\text{col}}\, k_{\text{row}}^{\top}\right) = \left(X * k_{\text{row}}\right) * k_{\text{col}}$$

- Box and triangle masks are outer products of 1D masks
- Filter the rows, then the columns: $2K$ instead of $K^2$ operations per pixel
- With the exercise classes: reshape, filter, transpose, filter again
- Gaussian and Sobel masks are separable too

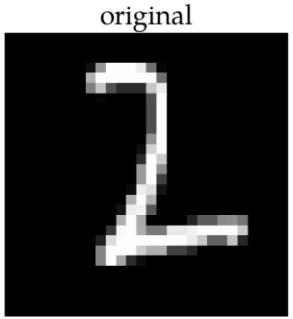


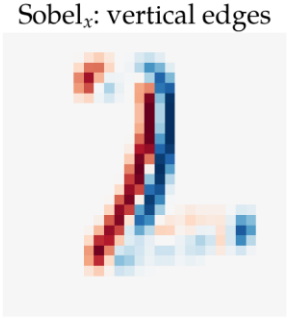

```
tri = CConvKernelTriangle(kernel_size=5)
img = x.reshape(28, 28)
# 1) rows: the 1D kernel on the flattened image
rows = tri.kernel(img.ravel()).reshape(28, 28)
# 2) columns: transpose, filter, transpose back
out = tri.kernel(rows.T.ravel()).reshape(28, 28).T
```

# Beyond Smoothing: Edge Detection

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}, \qquad S_y = S_x^\top$$

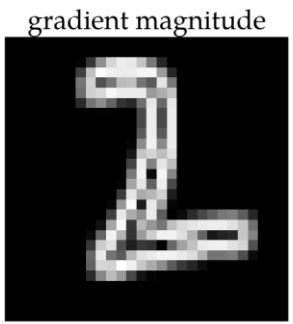
- [−1 0 1] sums to zero: no response on flat regions, strong response on edges
- Sobel = triangle smoothing in one direction × derivative in the other
- Red/blue: positive/negative responses (the sign depends on the flip)
- CNNs: the same operation, with masks learned from data









# Extras

Efficient implementations and additional exercises

# Efficient Implementation

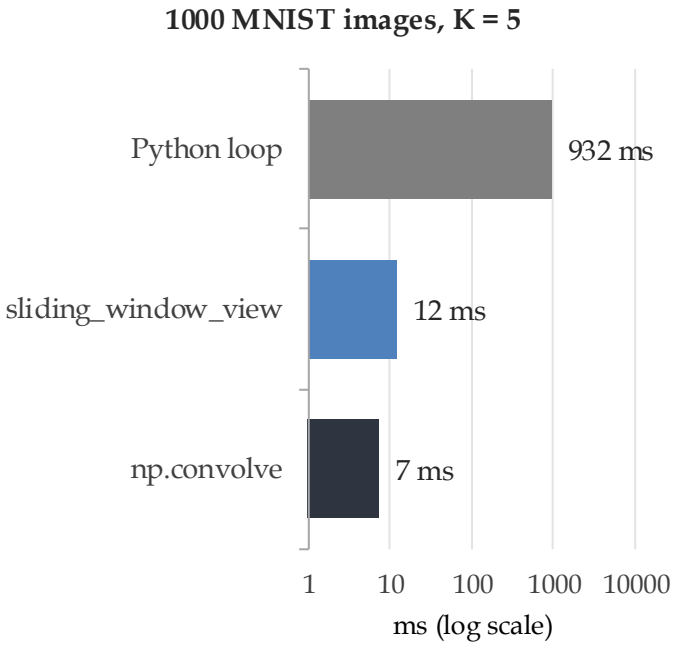
```python
from numpy.lib.stride_tricks import sliding_window_view
h = k.size // 2
xp = x.copy()

# 1) Python loop: one output at a time
for i in range(h, x.size - h):
    xp[i] = np.dot(k, x[i - h:i + h + 1])

# 2) all windows at once: (N-K+1) x K matrix
xp[h:-h] = sliding_window_view(x, k.size) @ k

# 3) NumPy convolution (it flips k: pass k[::-1])
xp[h:-h] = np.convolve(x, k[::-1], mode='valid')
```

Same output, 77–129× faster without the Python loop over samples



# Additional Programming Exercises

- Create an abstract class for 2D kernels with $K \times K$ masks
  - implement 2D moving average and triangle; check them against rows + columns filtering
- Implement `kernel()` with `sliding_window_view` and `np.convolve`; compare running times with the loop
- Verify numerically that:
  - the impulse response of a combo equals `np.convolve` of its masks
  - changing the order of the kernels in a combo only affects the borders
- Implement convolution with the FFT (zero-padding to $N + K - 1$) and compare it with `np.convolve`
- Perturb the MNIST test digits with Gaussian noise: does smoothing before the NMC classifier reduce the test error?